Practical 3:

Part A

Problem Statement:

1. Distinct points (Point.java + Driver.java with main)

(a) Create class Point with private int x, y and a constructor.

(b) Override toString() to return “(x, y)”.

(c) Override equals(Object) to be true when both x and y match, and hashCode() to agree

(e.g. Objects.hash(x, y)).

(d) In Driver.main: build a Point[] with some repeated coordinates.

(e) Count distinct points: for each point, use equals() to check whether it already

appeared earlier; count those that did not.

(f) Print “Distinct: N”.

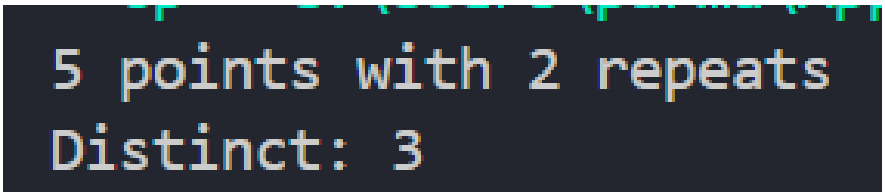
Expected output: 5 points with 2 repeats → “Distinct: 3”.

Solution:

```
import java.util.Objects;
class Point{
   private int x,y;
   public Point(int x,int y){
      this.x=x;
      this.y=y;
   }
   @Override
   public String toString(){
      return"("+x+", "+y+")";
   }
   @Override
   public boolean equals(Object obj){
      if(this==obj)return true;
```

```
    if(obj==null||getClass()!=obj.getClass())return false;
    Point p=(Point)obj;
    return x==p.x&&y==p.y;
  }
  @Override
  public int hashCode(){
    return Objects.hash(x,y);
  }
}
public class Driver1{
  public static void main(String[]args){
    Point[]points={new Point(1,2),new Point(3,4),new Point(5,6),new Point(1,2),new
Point(3,4)};
    int distinct=0;
    for(int i=0;i<points.length;i++){
      boolean isDuplicate=false;
      for(int j=0;j<i;j++){
        if(points[i].equals(points[j])){
          isDuplicate=true;
          break;
        }
      }
      if(!isDuplicate){
        distinct++;
      }
    }
    System.out.println("Distinct: "+distinct);
  }
}
```

Output:



Problem Statement:

2. Duplicate card check (Card.java + Driver.java)

(a) Create class Card with private String rank, suit and a constructor.

(b) Override toString() to return rank + " of " + suit (e.g. "Queen of Hearts").
(c) Override equals()/hashCode() by rank + suit.
(d) In Driver.main: add cards to a Card[] one at a time; before adding each, use equals()
to check it against the earlier cards.
(e) Print "Duplicate found: <card>" the first time a repeat appears.
Expected output: a second "Ace of Spades" → "Duplicate found: Ace of Spades".

Solution:

```
import java.util.Objects;
class Card{
   private String rank,suit;
   public Card(String rank,String suit){
      this.rank=rank;
      this.suit=suit;
   }
   @Override
   public String toString(){
      return rank+" of "+suit;
   }
   @Override
   public boolean equals(Object obj){
      if(this==obj)return true;
      if(obj==null||getClass()!=obj.getClass())return false;
      Card c=(Card)obj;
      return rank.equals(c.rank)&&suit.equals(c.suit);
   }
   @Override
   public int hashCode(){
      return Objects.hash(rank,suit);
   }
}
public class Driver2{
   public static void main(String[]args){
      Card[]cards=new Card[5];
```

```
        Card[]incoming={new Card("Ace","Spades"),new
Card("King","Hearts"),new Card("Queen","Diamonds"),new
Card("Ace","Spades")};
        int size=0;
        for(int i=0;i<incoming.length;i++){
            boolean duplicate=false;
            for(int j=0;j<size;j++){
                if(incoming[i].equals(cards[j])){
                    System.out.println("Duplicate found: "+incoming[i]);
                    duplicate=true;
                    break;
                }
            }
            if(duplicate)break;
            cards[size++]=incoming[i];
        }
    }
}
```

Output:

```
Duplicate found: Ace of Spades
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3>
```

Part B — Project Work (Hour 2): MiniBank

Problem Statement:

In Practical 2 you created the Account and Customer classes. Here you make their objects
behave correctly — they print as readable text (toString) and are compared by account
number (equals/hashCode) — and you add a nested Address class. Correct equals/hashCode
is essential because Practical 12 will store accounts in collections that rely on it.

1. In Account, override the toString() method to return a readable line containing
accountNumber, ownerName and balance.
2. In Account, override equals(Object o) and hashCode() so that two accounts are equal
when their accountNumber values are equal.
3. In Customer, add a public static nested class named Address with String fields line, city
and pincode plus getters; add an Address field to Customer and a getAddress() method.
4. In Customer, add a clone() method (implement Cloneable) that returns a copy of the
customer.
5. In main, print accounts using toString(), compare two Account objects with equals(), and
use instanceof to check an object's type.

Solution:

```
import java.util.Objects;
import java.util.Scanner;

class Customer implements Cloneable {

   private String name;
   private String email;
   private String mobile;
   private final String customerId;
   private Address address;

   private static long customerCounter = 101;

   public static class Address {
      private String line;
      private String city;
      private String pincode;

      public Address(String line, String city, String pincode) {
```

```
        this.line = line;
        this.city = city;
        this.pincode = pincode;
    }

    public String getLine() {
        return line;
    }

    public String getCity() {
        return city;
    }

    public String getPincode() {
        return pincode;
    }
}

private static String generateCustomerId() {
    return "CUST" + customerCounter++;
}

public Customer(String name, String email, String mobile, Address address) {
    this.name = name;
    this.email = email;
    this.mobile = mobile;
    this.address = address;
    this.customerId = generateCustomerId();
}

public String getName() {
    return name;
}

public String getEmail() {
    return email;
}

public String getMobile() {
```

```
        return mobile;
    }

    public String getCustomerId() {
        return customerId;
    }

    public Address getAddress() {
        return address;
    }

    @Override
    public Customer clone() {
        try {
            return (Customer) super.clone();
        } catch (CloneNotSupportedException e) {
            return null;
        }
    }
}

class Account {

    private final String accountNumber;
    private String ownerName;
    private long balance;
    private boolean active;

    private static int accountCounter = 1;

    private static String generateAccountNumber() {
        return String.format("AC%04d", accountCounter++);
    }

    public Account(String ownerName, long balance) {
        this.accountNumber = generateAccountNumber();
        this.ownerName = ownerName;
        this.balance = balance;
        this.active = true;
```

```
}

public Account(String ownerName) {
   this(ownerName, 0);
}

public void deposit(long amount) {
   if (amount > 0) {
      balance += amount;
   }
}

public boolean withdraw(long amount) {
   if (amount > 0 && balance >= amount) {
      balance -= amount;
      return true;
   }
   return false;
}

public String getAccountNumber() {
   return accountNumber;
}

public String getOwnerName() {
   return ownerName;
}

public long getBalance() {
   return balance;
}

public boolean isActive() {
   return active;
}

@Override
public String toString() {
   return accountNumber + " | " + ownerName + " | Balance = ₹" + balance;
```

```
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj)
            return true;

        if (obj == null || getClass() != obj.getClass())
            return false;

        Account a = (Account) obj;
        return accountNumber.equals(a.accountNumber);
    }

    @Override
    public int hashCode() {
        return Objects.hash(accountNumber);
    }
}

public class MiniBank {

    record BankInfo(String name, String branch) {
    }

    enum MenuOption {
        OPEN_ACCOUNT,
        DEPOSIT,
        WITHDRAW,
        TRANSFER,
        EXIT
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        BankInfo bank = new BankInfo("MiniBank", "Main Branch");
```

```
Customer.Address address =
        new Customer.Address("12 MG Road", "Ahmedabad", "380001");

Customer c1 = new Customer(
        "Vaidehi",
        "vaidehi@gmail.com",
        "1234567890",
        address);

Customer copy = c1.clone();

Account[] accounts = {
        new Account(c1.getName(), 5000),
        new Account("Riya"),
        new Account("Rahul", 10000)
};

System.out.println("Customer Details");
System.out.println("ID: " + c1.getCustomerId());
System.out.println("Name: " + c1.getName());
System.out.println("Email: " + c1.getEmail());
System.out.println("Mobile: " + c1.getMobile());
System.out.println("City: " + c1.getAddress().getCity());

System.out.println("\nTransactions");

accounts[0].deposit(1000);
accounts[1].deposit(3000);

System.out.println("Withdraw ₹2000 from Account 1: "
        + accounts[0].withdraw(2000));

System.out.println("Withdraw ₹4000 from Account 3: "
        + accounts[2].withdraw(4000));

System.out.println("\nAccount Details");

for (Account acc : accounts) {
   System.out.println(acc);
```

```
}

System.out.println("\nEquals Demo");

System.out.println(accounts[0].equals(accounts[1]));
System.out.println(accounts[0].equals(accounts[0]));

System.out.println("\nInstanceof Demo");

Object obj = accounts[0];

if (obj instanceof Account) {
   System.out.println("Object is an Account.");
}

System.out.println("\nClone Demo");

System.out.println("Original Customer : " + c1.getName());
System.out.println("Cloned Customer   : " + copy.getName());

System.out.println("\n================================");
System.out.println(bank.name() + " - " + bank.branch());
System.out.println("================================");

while (true) {

   System.out.println("\nMenu");
   System.out.println("1. Open Account");
   System.out.println("2. Deposit");
   System.out.println("3. Withdraw");
   System.out.println("4. Transfer");
   System.out.println("5. Exit");

   System.out.print("Enter your choice: ");

   int choice = sc.nextInt();

   MenuOption option = switch (choice) {
      case 1 -> MenuOption.OPEN_ACCOUNT;
```

```
            case 2 -> MenuOption.DEPOSIT;
            case 3 -> MenuOption.WITHDRAW;
            case 4 -> MenuOption.TRANSFER;
            case 5 -> MenuOption.EXIT;
            default -> null;
        };

        if (option == null) {
            System.out.println("Invalid choice!");
            continue;
        }

        switch (option) {

            case OPEN_ACCOUNT ->
                System.out.println("Open Account - To be implemented in a later lab.");

            case DEPOSIT ->
                System.out.println("Deposit - To be implemented in a later lab.");

            case WITHDRAW ->
                System.out.println("Withdraw - To be implemented in a later lab.");

            case TRANSFER ->
                System.out.println("Transfer - To be implemented in a later lab.");

            case EXIT -> {
                System.out.println("Thank you for using MiniBank!");
                sc.close();
                return;
            }
        }
    }
  }
}
```

Output:

```
Customer Details
ID: CUST101
Name: Meet
Email: meet@gmail.com
Mobile: 9876543210
City: Ahmedabad

Transactions
Withdraw ?2000 from Account 1: true
Withdraw ?4000 from Account 3: true

Account Details
AC0001 | Meet   | Balance = ?4000
AC0002 | Riya   | Balance = ?30000
AC0003 | Rahul  | Balance = ?60000

Equals Demo
false
true

Instanceof Demo
Object is an Account.

Clone Demo
Original Customer : Meet
Cloned Customer   : Meet

================================
MiniBank - Main Branch
================================

Menu
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Exit
Enter your choice: 5
Thank you for using MiniBank!
```